

TP 3 : Fichiers et graphiques MATLAB

Ali ZAINOUL
ali.zainoul.az@gmail.com

12 novembre 2025

Objectifs pédagogiques :

- Apprendre à importer et exporter des données avec les fonctions MATLAB intégrées.
- S'exercer aux techniques de tracé basiques et avancées pour des données vectorielles et matricielles.
- Manipuler les propriétés des figures et sauvegarder des graphiques dans différents formats.
- Développer des scripts pour automatiser des workflows de visualisation.

Consignes : Lisez chaque section, étudiez les exemples, puis réalisez les exercices. Commentez chaque étape dans vos scripts.

1. Importation et exportation de données

MATLAB propose plusieurs fonctions pour lire et écrire des fichiers. Choisissez la fonction en fonction du type de fichier et du flux de travail souhaité.

Fonction	Format	Description
<code>readtable</code>	CSV, XLSX, TXT	Retourne des données sous forme de table avec noms de variables.
<code>writetable</code>	CSV, XLSX, TXT	Écrit une table dans un fichier.
<code>csvread</code> / <code>csvwrite</code>	CSV	Lecture/écriture numérique (ancien).
<code>load</code> / <code>save</code>	MAT	Charger/enregistrer des variables d'espace de travail.
<code>importdata</code>	TXT, CSV	Import flexible pour données mixtes.

TABLE 1 – Fonctions courantes d'import/export

Exemple : importer un fichier CSV

```

1 % Suppose 'data.csv' with header row and numeric data
2 t = readtable('data.csv');
3 % Inspect first few rows
4 head(t);
5
6 year = t.Year;
7 value = t.Value;

```

Exemple : exporter des données traitées

```

1 % Create summary table
2 summary_table = table(year, value, 'VariableNames', {'Year','Value'});
3 % Write to new file
4 writetable(summary_table, 'summary.csv');

```

2. Tracés basiques

Visualisez des vecteurs et des tables en utilisant les fonctions de tracé standards.

2.1 Tracé en ligne

```

1 % Simple time series
2 x = year;
3 y = value;
4 figure;
5 plot(x, y, '-o');
6 xlabel('Year');
7 ylabel('Value');
8 title('Yearly Values');
9 legend('Value','Location','best');
10 grid on;

```

2.2 Courbes multiples

```
1 % Compare two series
2 y2 = value * 1.1; % dummy variation
3 figure;
4 plot(x, y, '-o', x, y2, '--s');
5 legend('Original','Variant');
```

2.3 Subplots

```
1 figure;
2 subplot(2,1,1);
3 plot(x, y);
4 title('Original');
5 subplot(2,1,2);
6 plot(x, y2);
7 title('Variant');
```

3. Visualisation avancée

Explorez d'autres types de tracés et la personnalisation des figures.

3.1 Histogramme et diagramme en barres

```
1 figure;
2 bar(x, y);
3 title('Bar Chart');
4
5 figure;
6 histogram(value, 10);
7 title('Histogram of Values');
```

3.2 Nuage de points et heatmap

```
1 % Scatter plot of two variables
2 z = rand(size(y));
3 figure;
4 scatter(y, z, 50, y, 'filled');
5 colorbar;
6 title('Scatter colored by Value');
7
8 % Heatmap of correlation matrix
9 C = corrcoef([y, z]);
10 figure;
11 heatmap(C, 'XDisplayLabels', {'Y','Z'}, 'YDisplayLabels', {'Y','Z'});
12 title('Correlation Heatmap');
```

4. Exportation de figures

Sauvegardez les figures pour des rapports ou des présentations.

```

1 fig = gcf;
2 % Save as PNG
3 saveas(fig, 'myplot.png');
4 % High-resolution PDF
5 print(fig, 'myplot.pdf', '-dpdf', '-r300');
6 % Newer: exportgraphics (vector)
7 exportgraphics(fig, 'myplot.svg', 'ContentType','vector');

```

5. Exercices

Créez des scripts ou fonctions pour automatiser les tâches suivantes. Commentez vos choix.

Exercice 1 : Importer et tracer une série temporelle

Objectif : Lire `population.csv` avec les colonnes `Year` et `Population`. Tracer `Population` en fonction de `Year` et sauvegarder la figure en PNG.

Exercice 2 : Comparer plusieurs séries

Objectif : Importer deux fichiers CSV : `series1.csv` et `series2.csv`. Tracer les deux séries sur les mêmes axes avec des marqueurs différents, ajouter une légende et une grille.

Exercice 3 : Statistiques résumées et export

Objectif : Lire `data.csv`, calculer la moyenne et l'écart type pour chaque colonne numérique, et sauvegarder les résultats dans `stats.xlsx`.

Exercice 4 : Mise en page personnalisée

Objectif : Créer une figure en 2x2 contenant : un tracé en ligne, un diagramme en barres, un histogramme et un nuage de points, en utilisant des vecteurs d'exemple définis dans votre script.

Exercice 5 : Fonction de reporting automatisée

Objectif : Écrire une fonction `create_report(data_file, out_fig)` qui :

- importe `data_file` (CSV),
- trace les deux premières colonnes,
- sauvegarde la figure sous `out_fig`,
- retourne une structure avec les champs : `mean_vals`, `std_vals`.

Exercice 6 : Manipulation de fichiers MAT

Objectif : S'exercer à sauvegarder et charger des données en format `.mat`, puis modifier et ré-enregistrer le fichier.

1. Dans un script, créez deux variables :

```
1  time = 0:0.1:2*pi;
2  signal = sin(time);
3
4  % Save both variables to a MAT-file
5  save('signal_data.mat', 'time', 'signal');
6
7  % Clear workspace to simulate a fresh session
8  clear;
9
10 % Load the saved data
11 data = load('signal_data.mat');
12 time = data.time;
13 signal = data.signal;
14
15 % Modify the signal: compute its envelope
16 envelope = abs(hilbert(signal));
17
18 % Add the new variable to the MAT-file without overwriting existing ones
19 save('signal_data.mat', 'envelope', '-append');
20
21 % Verify by reloading only the envelope variable
22 clear;
23 loaded_env = load('signal_data.mat', 'envelope');
24
25 % Plot original signal and envelope
26 figure;
27 plot(time, signal, 'b-', time, loaded_env.envelope, 'r--');
28 xlabel('Time (s)');
29 ylabel('Amplitude');
30 legend('Signal', 'Envelope');
31 title('Signal and Its Envelope');
32 grid on;
```

2. Dans des commentaires, expliquez chaque étape : sauvegarde, `clear`, chargement, ajout (`-append`) et tracé.
-

Corrections des exercices

Correction Exercise 1

```
1 % Script: Import and plot a time series from CSV
2 t = readtable('population.csv'); % Import CSV
3 year = t.Year;
4 pop = t.Population;
5
6 figure;
7 plot(year, pop, '-o');
8 xlabel('Year');
9 ylabel('Population');
10 title('Population over Years');
11 grid on;
12
13 % Save figure as PNG
14 saveas(gcf, 'population_plot.png');
```

Correction Exercise 2

```
1 % Script: Compare two series from CSV files
2 s1 = readtable('series1.csv');
3 s2 = readtable('series2.csv');
4
5 x = s1.Year;
6 y1 = s1.Value;
7 y2 = s2.Value;
8
9 figure;
10 plot(x, y1, '-o', x, y2, '--s');
11 xlabel('Year');
12 ylabel('Values');
13 legend('Series 1','Series 2','Location','best');
14 grid on;
```

Correction Exercise 3

```
1 % Script: Compute mean and std of numeric columns and export to XLSX
2 data = readtable('data.csv');
3 numeric_vars = varfun(@isnumeric, data, 'OutputFormat','uniform');
4
5 mean_vals = varfun(@mean, data(:,numeric_vars));
6 std_vals = varfun(@std, data(:,numeric_vars));
7
8 % Create summary table
9 summary_stats = table(mean_vals{1,:}', std_vals{1,:}', ...
10 'VariableNames', {'Mean','Std'});
11 writetable(summary_stats, 'stats.xlsx');
```

Correction Exercise 4

```
1 % Script: 2x2 subplots with different plot types
2 x = 1:10;
3 y = rand(1,10);
4
5 figure;
6
7 subplot(2,2,1);
8 plot(x,y,'-o');
9 title('Line Plot');
10
11 subplot(2,2,2);
12 bar(x,y);
```

```

13 title('Bar Chart');
14
15 subplot(2,2,3);
16 histogram(y,5);
17 title('Histogram');
18
19 subplot(2,2,4);
20 scatter(x,y,50,y,'filled');
21 colorbar;
22 title('Scatter Plot');

```

Correction Exercice 5

```

1 function report = create_report(data_file, out_fig)
2     % Import CSV data
3     T = readtable(data_file);
4     x = T(:,1);
5     y = T(:,2);
6
7     % Plot
8     fig = figure;
9     plot(x,y,'-o');
10    xlabel('X');
11    ylabel('Y');
12    title('Automated Report Plot');
13    grid on;
14
15    % Save figure
16    saveas(fig, out_fig);
17
18    % Compute statistics
19    report.mean_vals = mean(T(:,1:2));
20    report.std_vals = std(T(:,1:2));
21 end

```

Correction Exercice 6

```

1 % Script: Save, load, modify and append MAT-file
2 time = 0:0.1:2*pi;
3 signal = sin(time);
4
5 % Save variables
6 save('signal_data.mat', 'time', 'signal');
7
8 % Clear workspace
9 clear;
10
11 % Load saved data
12 data = load('signal_data.mat');
13 time = data.time;
14 signal = data.signal;
15
16 % Modify signal: compute envelope
17 envelope = abs(hilbert(signal));
18
19 % Append new variable
20 save('signal_data.mat', 'envelope', '-append');
21
22 % Verify by reloading only envelope
23 clear;
24 loaded_env = load('signal_data.mat', 'envelope');
25
26 % Plot original signal and envelope
27 figure;
28 plot(time, signal, 'b-', time, loaded_env.envelope, 'r--');
29 xlabel('Time (s)');
30 ylabel('Amplitude');
31 legend('Signal','Envelope');
32 title('Signal and Its Envelope');
33 grid on;

```